

Simulación Paginación y Segmentación

J. Feng Liu, F. Kuo Liu, and D. A. Alexander Vargas

Abstract—This paper presents the design and implementation of a concurrent multi-process simulation environment to evaluate and contrast memory management architectures under Paging and Pure Segmentation models. Developed in C for UNIX/Linux environments, the system comprises four independent software entities acting asynchronously within the user space: an Initializer, a Thread Producer, a passive Monitoring Spy, and a Finalizer. The core research addresses the synchronization and mutual exclusion paradigms inherent to shared hardware emulation, leveraging System V IPC shared memory segments (`shmget`) and POSIX named semaphores (`sem_open`) to secure critical regions. Performance analysis validates that while the Paging algorithm optimizes resource utilization through non-contiguous space allocation, the Pure Segmentation schema exhibits high vulnerability to severe external fragmentation under high-contention workloads, triggering process termination. Experimental results demonstrate absolute determinism and atomic state consistency across the global monitoring tables and chronological log files, validating the robustness of the implemented concurrency protocols.

I. INTRODUCCIÓN

El diseño y la gestión eficiente de la memoria principal representan uno de los pilares fundamentales en el desarrollo de los sistemas operativos modernos, requiriendo mecanismos robustos de asignación y sincronización de recursos compartidos. En este artículo, se presenta la implementación de un entorno simulado de gestión de memoria basado en los esquemas de paginación y segmentación pura empleando el lenguaje de programación C en un entorno de tipo UNIX/Linux. El núcleo de la investigación se centra en resolver la problemática de la sincronización y la exclusión mutua mediante el uso estratégico de semáforos interproceso y segmentos de memoria compartida (`shmget`). A través del análisis de las interacciones concurrentes entre un programa productor de hilos, una bitácora centralizada de eventos y una entidad de monitoreo pasivo, se evalúa el comportamiento de los procesos frente a la disponibilidad de espacio y la contención en regiones críticas. Los resultados obtenidos permiten contrastar la viabilidad técnica de ambos esquemas de traducción de direcciones, validando la estabilidad de los mecanismos de concurrencia aplicados.

II. INVESTIGACIÓN (MMAP VS SHMGET)

En sistemas operativos tipo UNIX/Linux, tanto `mmap` como `shmget` permiten compartir memoria entre procesos, pero pertenecen a modelos distintos de administración de memoria compartida.

La función `shmget`() forma parte del mecanismo de memoria compartida de System V IPC. Este método crea un segmento de memoria compartida identificado mediante una clave (*key*) administrada por el kernel del sistema operativo. Posteriormente, los procesos utilizan `shmat`() para adjuntar

dicho segmento a su espacio de direcciones virtuales. Una característica importante es que el segmento permanece activo hasta que se elimine explícitamente mediante `shmctl`() o hasta que el sistema se reinicie [1].

Por otro lado, `mmap`() pertenece al modelo POSIX y permite mapear archivos o regiones de memoria directamente en el espacio de direcciones de un proceso. Cuando se utiliza junto con la bandera `MAP_SHARED`, múltiples procesos pueden acceder simultáneamente a la misma región de memoria. Además, `mmap`() puede trabajar con archivos normales, memoria anónima (`MAP_ANONYMOUS`) o memoria compartida POSIX creada mediante `shm_open`() [2].

Una diferencia fundamental radica en el modelo de administración. `shmget`() utiliza el esquema tradicional de System V, considerado más antiguo y con una interfaz más compleja, mientras que `mmap`() sigue el estándar POSIX moderno, generalmente considerado más flexible y sencillo de utilizar. En sistemas Linux actuales, la memoria compartida POSIX suele implementarse internamente mediante la biblioteca de `mmap`() [1].

Asimismo, `mmap` ofrece ventajas adicionales, como la posibilidad de mapear archivos directamente en memoria, reduciendo operaciones de lectura y escritura tradicionales. Esto mejora la eficiencia en ciertas aplicaciones de alto rendimiento y facilita el intercambio de datos entre procesos independientes. En contraste, `shmget`() está orientado exclusivamente a memoria compartida administrada de manera interna por el kernel [3].

TABLE I
COMPARACIÓN ENTRE MMAP Y SHMGET

Característica	<code>mmap</code>	<code>shmget</code>
Estándar	POSIX	System V IPC
Tipo de memoria	Archivos o memoria compartida	Solo memoria compartida
Facilidad de uso	Más simple y flexible	Más complejo
Persistencia	Depende del mapeo y archivo	Persiste hasta eliminarse explícitamente
Funciones relacionadas	<code>mmap</code> , <code>munmap</code> , <code>shm_open</code>	<code>shmget</code> , <code>shmat</code> , <code>shmdt</code> , <code>shmctl</code>
Uso moderno	Recomendado en sistemas actuales	Compatibilidad con sistemas antiguos

III. TIPO DE SEMÁFORO UTILIZADO Y JUSTIFICACIÓN

Para la simulación del sistema de paginación y segmentación se requiere de una sincronización entre procesos de modo que se cuando se utilicen recursos compartidos estos se manejen de forma clara y sin riesgos de que estos procesos puedan afectarse entre sí. En la resolución del ejercicio se utilizaron tres semáforos binarios de tipo POSIX. La razón del porqué se usaron radica en la naturaleza del sistema, que consiste en cuatro programas diferentes, *inicializador.c*,

productor.c, *espia.c* y *finalizador.c*, en donde es necesaria una comunicación entre cada uno de los procesos de cada programa.

A diferencia de los semáforos anónimos POSIX creados con `sem_init`, los cuales solo funcionan entre hilos de un mismo proceso o entre procesos relacionados por herencia mediante `fork()`, los semáforos nombrados persisten en el sistema de archivos del kernel identificados por un nombre, lo que permite que cualquier proceso independiente pueda abrirlos mediante `sem_open()` usando dicho nombre. Esto los hace la opción natural para este proyecto, donde los cuatro programas se ejecutan en terminales separadas sin ninguna relación padre-hijo entre sí.

También se descartó el uso de semáforos System V (`semget/semop`), que si bien también permiten comunicación entre procesos independientes, presentan una API considerablemente más compleja y verbosa. Los semáforos nombrados POSIX logran el mismo objetivo con una interfaz más limpia y directa.

Se definieron tres semáforos, cada uno protegiendo un recurso compartido distinto: `/mutex_memoria` garantiza que únicamente un hilo a la vez ejecute el algoritmo de búsqueda o liberación de espacio en la RAM simulada; `/mutex_bitacora` serializa las escrituras concurrentes al archivo `bitacora.txt` evitando entrelazamiento de texto; y `/mutex_estado` protege la tabla de estados de hilos en memoria compartida, que el productor modifica y el espía consulta de forma concurrente. Los tres semáforos son inicializados en 1 por el inicializador, funcionando como mutex binarios, y son destruidos por el finalizador mediante `sem_unlink` al concluir la simulación.

IV. MECANISMO DE SINCRONIZACIÓN E IMPLEMENTACIÓN DE REGIONES CRÍTICAS

La arquitectura del sistema simulado se compone de múltiples procesos independientes concurrentes (Inicializador, Productor de Procesos, Espía y Finalizador) que interactúan dinámicamente sobre recursos compartidos en el espacio de núcleo. Debido a la naturaleza asíncrona de los hilos generados por el productor, la sincronización y la exclusión mutua se gestionan mediante el uso estratégico de semáforos POSIX nombrados (identificados como `/mutex_memoria`, `/mutex_bitacora` y `/mutex_estado`), coordinando el acceso a tres regiones críticas bien definidas:

A. Exclusión Mutua en la Asignación de Memoria Virtual

El recurso crítico principal es el segmento de memoria compartida de System V que simula la memoria RAM física. Tanto en el esquema de paginación como en el de segmentación pura, el algoritmo requiere inspeccionar la memoria (búsqueda de celdas vacías mediante *First-Fit*), modificar las celdas asignando el identificador del hilo (`IDx`), y posteriormente limpiar dichos bloques al despertar del letargo (*sleep*).

Si dos o más procesos ejecutaran estos algoritmos de búsqueda y asignación en paralelo, se produciría una condición de carrera (*race condition*) donde múltiples hilos podrían seleccionar e intentar escribir en las mismas celdas de memoria

de forma simultánea. Para mitigar este riesgo, el semáforo `sem_mutex` actúa como un cerrojo binario (*mutex*). El protocolo de sincronización sigue estrictamente la siguiente secuencia lineal de control:

- 1) **Solicitud de Entrada:** El hilo concurrente invoca la llamada al sistema `sem_wait(sem_mutex)`. Si el recurso está ocupado, el hilo es suspendido en ESTADO_BLOQUEADO.
- 2) **Región Crítica:** Al adquirir el cerrojo, cambia a ESTADO_BUSCANDO y ejecuta el algoritmo de asignación correspondiente (Paginación o Segmentación) en la memoria compartida.
- 3) **Liberación de Región:** El hilo invoca inmediatamente `sem_post(sem_mutex)`, permitiendo que otros hilos entren a la sección crítica de la memoria física.
- 4) **Fase de Letargo:** El hilo cambia a ESTADO_DURMIENDO e ingresa en un periodo de `sleep()` asíncrono.
- 5) **Desasignación Atómica:** Al despertar, el hilo repite los pasos 1, 2 y 3 exclusivamente para limpiar sus celdas asignadas y transicionar a ESTADO_TERMINADO.

Cada hilo se ve obligado a bloquear el mutex antes de iniciar la exploración. Si la región crítica está ocupada, el hilo pasa a un estado de bloqueo gestionado por el planificador del sistema operativo, garantizando así la exclusión mutua sobre la simulación de la RAM.

B. Sincronización de la Bitácora Centralizada de Eventos

El archivo de texto físico `bitacora.txt` es un recurso compartido globalmente por todos los hilos para registrar de manera cronológica las operaciones del sistema. Dado que las llamadas de E/S estándar de C (`fprintf`, `fopen`) no son atómicas a nivel de subprocesos, las escrituras concurrentes provocarían el entrelazado corrupto de líneas de texto o fallas en los descriptores de archivo.

La exclusión mutua en este subsistema se logra mediante la abstracción encapsulada en la función `escribirBitacora`, la cual utiliza el semáforo `sem_bitacora` para aislar el flujo de datos:

```
void escribirBitacora(const char* formato, ...) {
    sem_wait(sem_bitacora); // Entrada a RC
    FILE *f = fopen("bitacora.txt", "a");
    if (f) {
        time_t t = time(NULL);
        struct tm *tm_info = localtime(&t);
        fprintf(f, "[%02d:%02d:%02d] ",
                tm_info->tm_hour,
                tm_info->tm_min,
                tm_info->tm_sec);
        va_list args;
        va_start(args, formato);
        vfprintf(f, formato, args);
        va_end(args);
        fprintf(f, "\n");
        fclose(f);
    }
    sem_post(sem_bitacora); // Salida de RC
}
```

Este diseño garantiza que un solo hilo mantenga abierto el flujo de escritura del archivo a la vez, preservando la integridad del historial analítico y cronológico de la auditoría.

C. Coherencia en la Tabla de Estados Global

Para permitir que el programa *Espía* refleje pasivamente y en tiempo real el estado exacto de los procesos (identificando dinámicamente qué hilos están durmiendo, buscando espacio, bloqueados o muertos), se diseñó una estructura de datos global denominada *TablaEstado* montada sobre un segundo segmento de memoria compartida independiente (*KEY_ESTADO*).

El semáforo *sem_estado* coordina las operaciones concurrentes sobre esta tabla. Mientras los hilos del productor modifican secuencialmente sus propios registros mutando sus banderas de estado (por ejemplo, mediante *actualizarEstado*), el programa *espía* puede leer de forma segura dicha estructura acoplándose en modo de solo lectura (*SHM_RDONLY*), evitando lecturas sucias o estados inconsistentes durante la ejecución de la simulación.

V. PLANTEAMIENTO DEL PROBLEMA

El problema consiste en implementar una simulación de asignación de memoria a procesos mediante los esquemas de paginación y segmentación, aplicando mecanismos de sincronización para garantizar el acceso seguro a los recursos compartidos. El sistema debe ser implementado en C y estar compuesto por cuatro programas independientes que se comunican entre sí mediante memoria compartida y semáforos.

La memoria compartida, solicitada mediante *shmget*, representa la RAM del sistema y es el recurso central que los procesos compiten por obtener. Dado que los programas son procesos completamente independientes entre sí, sin ninguna relación de herencia, se requiere un mecanismo de sincronización que permita la comunicación entre ellos, razón por la cual se optó por semáforos nombrados POSIX, como se detalló en una sección anterior.

El sistema opera bajo la siguiente dinámica: el programa *inicializador* prepara el ambiente creando la memoria compartida, los semáforos y la bitácora de eventos. El programa *productor* genera hilos que representan procesos llegando al estado *Ready*, cada uno compitiendo por espacio en memoria bajo uno de los dos esquemas soportados:

- **Paginación:** cada hilo solicita entre 1 y 10 páginas de memoria, asignadas de forma no contigua.
- **Segmentación (First-Fit):** cada hilo solicita entre 1 y 5 segmentos, cada uno de entre 1 y 3 unidades contiguas de memoria.

La lógica de cada hilo sigue la siguiente secuencia de pasos:

- 1) Solicitar el semáforo de memoria (*sem_wait*).
- 2) Buscar espacio disponible en la RAM simulada según el algoritmo activo.
- 3) Registrar la acción en la bitácora.
- 4) Liberar el semáforo de memoria (*sem_post*).
- 5) Dormir el tiempo asignado (*sleep*).
- 6) Solicitar nuevamente el semáforo de memoria.
- 7) Liberar el espacio de memoria ocupado.
- 8) Registrar la desasignación en la bitácora.
- 9) Liberar el semáforo de memoria.

Si al momento de buscar espacio no existe suficiente memoria disponible, el hilo muere y registra este hecho en

la bitácora. Solo un hilo a la vez puede ejecutar el algoritmo de búsqueda, garantizando que dos procesos no seleccionen el mismo espacio de forma simultánea.

El programa *espía* permite observar en tiempo real el estado de la memoria y de los procesos sin interferir con la simulación, consultando directamente la memoria compartida en modo de solo lectura. Finalmente, el programa *finalizador* detiene la producción de nuevos hilos, destruye los segmentos de memoria compartida, elimina los semáforos y cierra la bitácora de eventos.

Todos los eventos relevantes quedan registrados en el archivo *bitacora.txt*, incluyendo asignaciones, desasignaciones y muertes de procesos, con su respectiva marca de tiempo y los espacios de memoria involucrados.

VI. RESULTADOS

En este apartado se documentan los casos de prueba, donde se incluye la explicación de qué se quería probar, cómo se hizo y el resultado del mismo. Además se adjunta un análisis de resultados, indicando qué funciona y qué no funciona, aparte de otros detalles relevantes. Finalmente, incluye un resumen de nuestras experiencias durante la realización del proyecto.

A. Análisis de Resultados

El desarrollo y ejecución del sistema de simulación multiproceso permitieron evaluar de forma empírica el impacto arquitectónico y operativo de los esquemas de asignación de memoria por Paginación y Segmentación Pura en un entorno de núcleo compartido de Linux. A continuación, se presenta el análisis detallado del rendimiento de los componentes del sistema, la eficiencia en la gestión de recursos compartidos y la efectividad de las primitivas de sincronización POSIX implementadas.

B. Evaluación del Rendimiento y Funcionalidad del Sistema

El sistema cumplió satisfactoriamente con el 100% de las operaciones lógicas y funcionales requeridas en la especificación del problema.

- **Lo que funciona correctamente:** La creación e inicialización del entorno a través de llamadas al sistema System V (*shmget*, *shmat*) se ejecuta de manera determinista. Los hilos concurrentes creados por el programa *Productor* interactúan con éxito sobre la memoria física emulada, aplicando exclusión mutua estricta mediante semáforos nombrados (*sem_open*). El programa *Espía* refleja de forma fidedigna y en tiempo real el vector global de estados (*ESTADO_BUSCANDO*, *ESTADO_DURMIENDO*, *ESTADO_BLOQUEADO*, *ESTADO_MUERTO* y *ESTADO_TERMINADO*).
- **Limitaciones operativas (Lo que no funciona o restricciones):** Debido a la naturaleza no bloqueante y pasiva del programa *Espía*, este no puede predecir ni forzar el orden de asignación de memoria; únicamente actúa como un observador acoplado en modo lectura (*SHM_RDONLY*). Asimismo, bajo el esquema de segmentación pura, se evidenció que la memoria es altamente susceptible a fallas catastróficas de asignación si la tasa de generación de hilos es alta y el tiempo de letargo (*sleep*) es prolongado.

C. Análisis Comparativo: Paginación vs. Segmentación Pura

La simulación arrojó contrastes drásticos entre ambos esquemas de administración de memoria virtual bajo las mismas condiciones de carga (tiempo aleatorio de generación de procesos entre 30s y 60s):

- 1) **Fragmentación y Desperdicio:** En el modo de *Paginación*, el aprovechamiento de la memoria compartida fue del 100% de las páginas libres, dado que las celdas asignadas a un proceso no requieren ser físicamente contiguas. El hilo localiza de forma atómica cualquier hueco disponible mediante un mapeo disperso. Por el contrario, en el modo de *Segmentación Pura*, utilizando el algoritmo de búsqueda *First-Fit* contiguo, el fenómeno de la **fragmentación externa** degradó el sistema con rapidez. Se observaron casos críticos donde la memoria total libre sumaba espacio suficiente, pero al estar fragmentada en bloques aislados de 1 unidad, provocó la muerte inmediata de hilos que solicitaban segmentos contiguos de tamaño 2 o superior.
- 2) **Ciclo de Vida y Mortalidad de Procesos:** En los escenarios de alta concurrencia, la tasa de mortalidad de los hilos se elevó sustancialmente bajo el régimen de segmentación. Como lo dicta el requerimiento del proyecto, si un proceso no encuentra espacio contiguo para la totalidad de sus segmentos definidos al azar (de 1 a 5 segmentos), este aborta su ejecución y transiciona directamente al estado MUERTO, registrando el fallo en la bitácora física.

VII. CASOS DE PRUEBA

Para validar el correcto funcionamiento de los algoritmos implementados, la gestión de la exclusión mutua mediante semáforos y el comportamiento dinámico de los hilos de ejecución, se diseñaron y ejecutaron cuatro casos de prueba fundamentales.

A. Caso 1: Inicialización del Entorno y Creación de Recursos IPC

Objetivo	Verificar que el programa Inicializador cree correctamente los segmentos de memoria compartida (RAM simulada y Tabla de Estados) y los semáforos nombrados, registrando los identificadores en el archivo de control.
Entrada	Ejecución del comando <code>./inicializador</code> . Se ingresa un tamaño para la memoria compartida equivalente a 50 celdas de simulación.
Resultado Esperado	Creación exitosa de los <code>shmids</code> para RAM y Estado vía <code>shmget</code> . Apertura e inicialización a 1 de los semáforos nombrados <code>/mutex_memoria</code> , <code>/mutex_bitacora</code> y <code>/mutex_estado</code> . Persistencia de estos IDs en <code>datos.txt</code> con la bandera <code>Producir: true</code> . El proceso inicializador termina tras cumplir la tarea.
Estado	Exitoso

B. Caso 2: Asignación de Memoria Compartida por Paginación

Objetivo	Comprobar que los hilos productores bajo el esquema de paginación adquieren el mutex, localizan celdas libres de forma no contigua, registran la asignación en la bitácora y entran en letargo de forma segura.
Entrada	Ejecución de <code>./productor</code> seleccionando el algoritmo <code>pag</code> . Un hilo generado de manera aleatoria solicita 4 páginas con un tiempo de simulación asignado de 30 segundos.
Resultado Esperado	El hilo adquiere el semáforo <code>sem_mutex</code> , mapea la RAM y encuentra 4 celdas libres (ej. celdas [0, 4, 7, 12]). Escribe su identificador <code>IDx</code> , registra el evento con marca de tiempo en <code>bitacora.txt</code> , libera el semáforo e invoca <code>sleep(30)</code> transicionando a <code>ESTADO_DURMIENDO</code> .
Estado	Exitoso

C. Caso 3: Asignación por Segmentación (First-Fit) y Muerte por Fragmentación

Objetivo	Evaluar el comportamiento del algoritmo de segmentación pura al buscar bloques contiguos y validar que el hilo sea destruido de forma segura liberando recursos parciales si no se satisface la condición total.
Entrada	Ejecución de <code>./productor</code> con el algoritmo <code>seg</code> . Un hilo solicita 3 segmentos con dimensiones de [3, 2, 2] unidades. La memoria compartida se encuentra severamente fragmentada y solo posee espacios contiguos máximos de 1 celda vacía.
Resultado Esperado	El hilo bloquea la región crítica y escanea la estructura con <i>First-Fit</i> . Al no hallar bloques contiguos suficientes, deshace cualquier asignación parcial previa en la RAM (limpieza atómica), escribe la entrada de tipo MUERTE en la bitácora y muta su bandera global a <code>ESTADO_MUERTO</code> .
Estado	Exitoso

D. Caso 4: Finalización de la Simulación y Liberación de Recursos

Objetivo	Garantizar que el programa Finalizador detenga la producción de hilos de forma asíncrona, remueva los segmentos de memoria compartida del sistema operativo y desvincule los semáforos del kernel.
Entrada	Ejecución del comando <code>./finalizador</code> en una terminal secundaria mientras el programa productor e hilos remanentes se encuentran activos y durmiendo en escena.
Resultado Esperado	Modificación inmediata de la bandera en <code>datos.txt</code> a <code>Producir: false</code> , deteniendo el bucle del productor en su próxima iteración. Destrucción de los segmentos de memoria mediante <code>shmctl(IPC_RMID)</code> . Ejecución de <code>sem_unlink</code> para todos los semáforos nombrados y adición del cierre final en la bitácora.
Estado	Exitoso

VIII. RESUMEN DE EXPERIENCIAS

El desarrollo de este proyecto representó un reto en nuestra formación como ingenieros en computación, permitiéndonos trasladar los conceptos teóricos de la administración de memoria virtual y la concurrencia directamente al diseño de software a bajo nivel en entornos Unix/Linux. A continuación, se presentan las principales experiencias, desafíos superados y lecciones aprendidas durante este proceso:

- 1) **Curva de Aprendizaje de Sincronización:** Una de las primeras experiencias impactantes fue la asimilación práctica de la diferencia entre hilos (*threads*) y procesos independientes de Linux. Comprender que los semáforos estándar de `pthread` no funcionan para coordinar programas ejecutados de manera separada nos obligó a investigar a fondo las llamadas al sistema para semáforos POSIX nombrados (`sem_open`, `sem_unlink`). La correcta manipulación de estos recursos del kernel requirió un diseño meticuloso para evitar condiciones de carrera antes de que el escenario estuviera completamente montado.
- 2) **El Desafío de Depurar la Concurrencia:** A diferencia del software secuencial convencional, la naturaleza asíncrona de los hilos generados por el productor hizo que la depuración (*debugging*) fuera una tarea compleja. Experimentamos bloqueos mutuos (*deadlocks*) en las primeras etapas debido a la inversión inadvertida del orden en la solicitud de los semáforos de la memoria y la bitácora. Aprendimos que el uso de herramientas tradicionales de depuración es limitado en entornos concurrentes, lo que nos llevó a valorar el diseño de protocolos de exclusión mutua antes de proceder con la codificación.
- 3) **Visualización Crítica de la Fragmentación Estructural:** Implementar el algoritmo de búsqueda *First-Fit* contiguo para la segmentación pura nos brindó una perspectiva real sobre el costo del desperdicio de memoria. Fue una experiencia valiosa observar a través del programa *Espía* cómo el sistema sufría de fragmentación externa en escenarios de estrés. Ver el comportamiento dinámico de hilos que eran rechazados y morían a pesar de que la memoria global libre superaba la cantidad solicitada (pero dispersa en bloques unitarios) consolidó de forma definitiva nuestra comprensión sobre las desventajas de la segmentación frente a la flexibilidad del esquema de paginación.
- 4) **Importancia de la Recolección de Basura en el Kernel:** Otra lección de gran relevancia técnica fue la persistencia de los recursos IPC (mecanismos de comunicación entre procesos) en Linux. Durante las pruebas iniciales, abortar abruptamente los procesos con `Ctrl+C` dejaba segmentos de memoria compartida (*shm*) y semáforos huérfanos bloqueando los recursos del sistema operativo. Esto nos ayudó a entender la importancia crítica del programa *Finalizador*, forzándonos a diseñar un mecanismo robusto de limpieza absoluta que invocara de forma segura las directivas `shmctl` (`IPC_RMID`) y `sem_unlink`.

En conclusión, la experiencia práctica evidenció que la programación de sistemas operativos exige una planificación lógica antes del desarrollo. El control de la concurrencia no tolera ambigüedades; cada recurso compartido, por mínimo que sea, debe estar estrictamente protegido, transformando la teoría abstracta de las lecciones magistrales en competencias de ingeniería de software tangibles y de alta fidelidad.

IX. MANUAL DE USUARIO

A. Requisitos

Para compilar y ejecutar el sistema se requiere un entorno Linux con `gcc` instalado y soporte para `pthread`. No se necesitan dependencias externas adicionales.

B. Compilación

Desde el directorio raíz del proyecto, ejecutar:

```
make
```

Esto generará los cuatro binarios: *inicializador*, *productor*, *espía* y *finalizador*. Para eliminar los binarios compilados:

```
make clean
```

C. Ejecución

Es importante respetar el orden de ejecución de los programas, ya que cada uno depende del ambiente preparado por el anterior.

1) *Paso 1 — Inicializar el ambiente:* En una terminal, ejecutar el inicializador. Este programa solicitará la cantidad de celdas de memoria a simular:

```
./inicializador
Ingrese el tamaño de la memoria compartida: 20
```

Este paso crea la memoria compartida, los tres semáforos nombrados y el archivo `bitacora.txt`. Los identificadores de los recursos quedan almacenados en `datos.txt` para que los demás programas puedan acceder a ellos. El inicializador termina su ejecución una vez completada la inicialización.

Nota: Si existe una sesión anterior sin finalizar correctamente, ejecutar primero `./finalizador` para limpiar los recursos residentes en el sistema operativo antes de volver a inicializar.

2) *Paso 2 — Iniciar el Productor:* En una **nueva terminal**, ejecutar el productor e indicar el algoritmo deseado:

```
./productor
Algoritmo (seg/pag): pag
```

El productor comenzará a generar hilos de forma periódica, con un intervalo aleatorio de entre 30 y 60 segundos. Cada hilo buscará espacio en memoria, dormirá si lo consigue, y luego liberará el espacio al terminar su ciclo. Este programa debe mantenerse en ejecución durante toda la simulación.

3) *Paso 3 — Observar con el Espía:* Con el productor corriendo, abrir **otra terminal** y ejecutar el espía:

```
./espia
```

El espía presenta un menú interactivo con las siguientes opciones:

```
--- MENU ESPIA ---
1. Ver estado de la memoria
2. Ver estado de los procesos
3. Ver bitacora
4. Salir
```

- **Opción 1:** Muestra el estado de cada celda de memoria, indicando si está ocupada o vacía y qué hilo la ocupa.
- **Opción 2:** Lista los hilos clasificados por estado: *durmiendo* (en memoria), *buscando* espacio, *bloqueados* (esperando semáforo), *muertos* (sin espacio) y *terminados* (ciclo completado).
- **Opción 3:** Imprime el contenido completo de `bitacora.txt`.

El espía accede a la memoria compartida en modo solo lectura, por lo que puede consultarse tantas veces como se desee sin interferir con la simulación.

4) *Paso 4 — Finalizar la simulación:* Cuando se desee detener el sistema, ejecutar el finalizador en cualquier terminal:

```
./finalizador
```

Este programa realiza las siguientes acciones en orden:

- 1) Cambia la bandera `Producir` a `false` en `datos.txt`, indicándole al productor que deje de generar hilos en su próxima iteración.
- 2) Destruye ambos segmentos de memoria compartida.
- 3) Elimina los tres semáforos nombrados del sistema.
- 4) Escribe el cierre en `bitacora.txt`.

D. Estructura de Archivos

```
p2-SO/
|-- comun.h          # Constantes, structs y
    estados compartidos
|-- inicializador.c   # Crea el ambiente
|-- productor.c       # Genera los hilos/procesos
|-- espia.c           # Observador interactivo
|-- finalizador.c     # Limpia y termina la
    simulacion
|-- Makefile          # Compilacion
|-- datos.txt         # Generado por inicializador
    (auto-generado)
|-- bitacora.txt      # Log de eventos
    (auto-generado)
```

X. CÓDIGO FUENTE Y REPOSITORIO

Con el propósito de garantizar la transparencia del diseño arquitectónico, la reproducibilidad de las pruebas concurrentes y permitir la auditoría de las llamadas al sistema implementadas (`shmget`, `sem_open`), el código fuente íntegro de los cuatro programas independientes, junto con los archivos de configuración y documentación complementaria, ha sido publicado bajo un sistema de control de versiones de acceso público.

El repositorio oficial del proyecto puede ser consultado y clonado a través de la siguiente dirección electrónica:

<https://github.com/djedrielle/p2-SO>

Este espacio físico contiene los archivos fuentes individuales (`inicializador.c`, `productor.c`, `espia.c` y `finalizador.c`), y el archivo de bitácora física generado durante las corridas de simulación del sistema.

REFERENCES

- [1] BowPark, "Linux shared memory: `shmget()` vs `mmap()`?" *Stack Overflow*, Jan. 23, 2014. Disponible en: <https://stackoverflow.com/questions/21311080/linux-shared-memory-shmget-vs-mmap>
- [2] Planck Legacy Archive, "SHMMAP," *Esa.int*, 2022. Disponible en: https://pla.esac.esa.int/ftp_public/plaavi/external_deps/temp/ust/local/harris/idl/help/online_help/Subsystems/idl/Content/Reference/%20Material/S/SHMMAP.htm?TocPath=Routines+%28alphabetical%29%7CRoutines%3A+S%7C_____29.
- [3] LinuxVox, "Where is Linux Shared Memory Actually Located? Physical vs Virtual Memory & Why Processes See Different Addresses," *linuxvox*, Dec. 05, 2025. Disponible en: <https://linuxvox.com/blog/where-is-linux-shared-memory-actually-located/>.

APPENDIX

La presente sección documenta de manera cronológica el desglose de actividades, la evolución del diseño lógico y el esfuerzo de desarrollo invertido en el proyecto durante los ciclos de control establecidos.

A. Semana 11: Del 4 al 10 de Mayo del 2026

Fecha	Descripción de Actividades e Hitos
Miércoles 6 de mayo	Análisis exhaustivo del enunciado del pliego de especificaciones del proyecto. Modelado lógico inicial de las dependencias de recursos IPC. Creación del repositorio del sistema de control de versiones y estructuración física de directorios del proyecto.
Jueves 7 de mayo	Creación del entorno base e inicio del desarrollo del <i>Programa Inicializador</i> . Se estructuró conceptualmente como una función modular dentro de una única arquitectura de código, generando dudas técnicas sobre si los programas debían convivir bajo hilos compartidos o como ejecutables separados del kernel.
Viernes 8 de mayo	Inicio de la codificación modular del <i>Programa Productor</i> . Ante la duda estructural previa, se recibe la aclaración docente: los 4 programas deben operar estrictamente como ejecutables binarios independientes que interactúan en caliente mediante el kernel. Se reestructura el código para separar la inicialización.
Sábado 9 de mayo	Implementación analítica y desarrollo de la lógica del <i>Productor</i> enfocada en el algoritmo de Paginación . Se diseñó el algoritmo de distribución no contigua de celdas libres en el bloque System V compartido mediante <code>shmget()</code> y <code>shmat()</code> .
Domingo 10 de mayo	Culminación de las pruebas preliminares de inserción de celdas por paginación. Diseño e inicio del desarrollo del <i>Programa Espía</i> , integrando el modo de solo lectura (<code>SHM_RDONLY</code>) para inspeccionar de forma segura la estructura física simulada sin alterar las regiones críticas.

B. Semana 12: Del 11 al 17 de Mayo del 2026

Fecha	Descripción de Actividades e Hitos
Lunes 11 de mayo	Desarrollo integral de la lógica del algoritmo de Segmentación Pura en el programa <i>Productor</i> . Codificación del algoritmo de asignación <i>First-Fit</i> secuencial y contiguo. Implementación de las validaciones de error y desasignación atómica para los procesos rechazados por falta de espacio.
Martes 12 de mayo	Integración de los semáforos POSIX nombrados (<code>sem_open</code> , <code>sem_wait</code> , <code>sem_post</code>) en las regiones críticas compartidas de memoria RAM, bitácora y tabla global de estados. Codificación del <i>Programa Finalizador</i> encargado de ejecutar la recolección de basura del sistema operativo mediante <code>shmctl(IPC_RMID)</code> y <code>sem_unlink</code> .
Miércoles 13 de mayo	Pruebas de integración multiproceso concurrentes. Conexión de la bitácora física centralizada garantizando la mutua exclusión en el flujo de escritura del descriptor de archivo <code>bitacora.txt</code> . Finalización del desarrollo total de los componentes lógicos programados.
Jueves 14 de mayo	Ejecución formal de los casos de prueba del sistema en condiciones de estrés y concurrencia masiva. Verificación cualitativa de los estados de los procesos visualizados en tiempo real a través del monitor interactivo del programa <i>Espía</i> .
Viernes 15 de mayo	Documentación técnica y redacción del artículo de investigación en el formato estándar de la plantilla <i>IEEE Transactions</i> . Elaboración de los cuadros comparativos entre los mecanismos <code>mmap</code> y <code>shmget</code> , y análisis formal de resultados.